# Draw It or Lose It

**CS 230 Project Software Design Template**
Version 3.0

**Table of Contents**


**CS 230 Project Software Design Template**

- **Table of Contents**
- **Document Revision History**
- **Executive Summary**
- **Requirements**
- **Design Constraints**
- **System Architecture View**
- **Domain Model**
- **Evaluation**
- **Recommendations**


## Document Revision History

| Version | Date | Author | Comments |
|---|---|---|---|
| 1.0 | | Hassaan Khan | Initial draft for Project One submission (template sections completed). |
| 2.0 | 03/01/26 | Hassaan Khan | Completed Evaluation table for Project Two (platform comparison). |
| 3.0 | 3/01/26 | Hassaan Khan | Completed Recommendations |

**Instructions**
Fill in all bracketed information on page one (the cover page), in the Document Revision History table, and below each header. Under each header, remove the bracketed prompt and write your own paragraph response covering the indicated information.

## Executive Summary

The Gaming Room wants to expand its existing Android game, Draw It or Lose It, into a web-based application that can serve multiple platforms. This design proposes a Java-based service layer that manages games, teams, and players using proven design patterns to enforce unique names and ensure only one game service instance exists in memory. The result is a scalable, maintainable foundation that can be hosted in a distributed environment and accessed by clients on desktop and mobile devices through standard web technologies.

## Requirements

Business requirements include expanding Draw It or Lose It to additional computing environments, supporting multiple teams and players per game, and providing a consistent user experience across platforms. Technical requirements include enforcing unique game and team names, assigning unique identifiers to games, teams, and players, and ensuring that only one instance of the GameService exists in memory. The solution must operate reliably in a web-based distributed environment where multiple users can access the application concurrently.

## Design Constraints

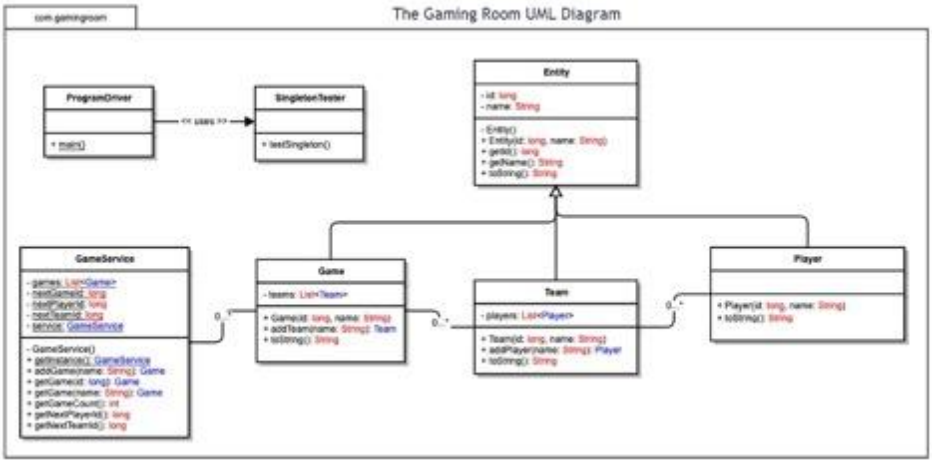
The application must be designed for a web-based distributed environment, which introduces constraints around network latency, client/server communication, and horizontal scaling. To support concurrent users, core services should be stateless where possible and rely on well-defined APIs for communication. Unique names for games and teams require consistent validation logic (implemented with iterator-based searches) to prevent duplicates, and unique identifiers must be centrally managed. Security is a key constraint because user data must be protected in transit and at rest, requiring strong authentication, authorization, and encrypted communication. Finally, platform independence and maintainability constrain the technology choices toward portable tools and frameworks that run consistently across environments.

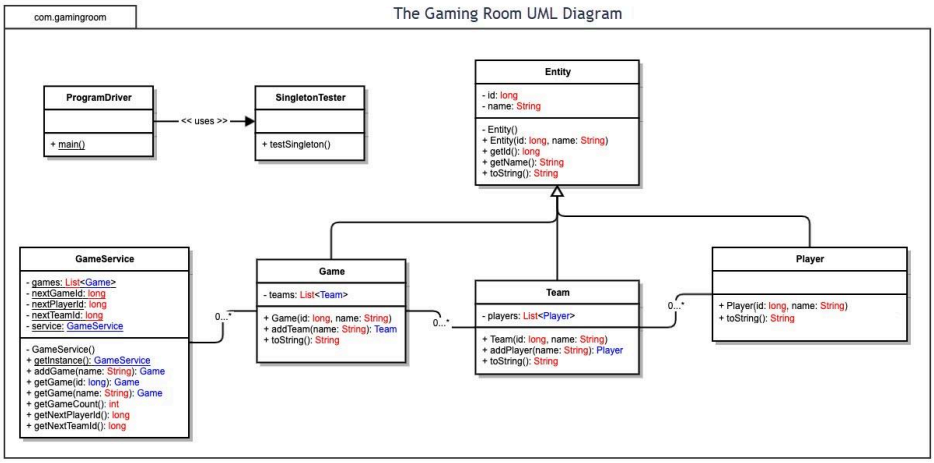
## System Architecture View

Please note: There is nothing required here for these projects, but this section serves as a reminder that describing the system and subsystem architecture present in the application, including physical components or tiers, may be required for other projects. A logical topology of the communication and storage aspects is also necessary to understand the overall architecture and should be provided.

## Domain Model

The UML domain model centers on a shared Entity base class that contains the common attributes id and name, which are inherited by Game, Team, and Player. This demonstrates inheritance and reuse while keeping shared data and behavior in one place. GameService is implemented as a singleton to ensure only one service instance manages the collections and identifier generation, which supports the requirement that only one instance of the game service exists in memory. A Game contains a list of Teams, and each Team contains a list of Players, showing has-a relationships and organization of the game state. Encapsulation is used by keeping fields private and exposing controlled methods such as addTeam and addPlayer. Iterator-based searches are used when adding games, teams, and players to enforce unique names efficiently and to retrieve instances by name or index.




## Evaluation

Using your experience to evaluate the characteristics, advantages, and weaknesses of each operating platform (Linux, Mac, and Windows) as well as mobile devices, consider the requirements outlined below and articulate your findings for each. As you complete the table,

keep in mind your client's requirements and look at the situation holistically, as it all has to work together.

In each cell, remove the bracketed prompt and write your own paragraph response covering the indicated information.

| **Development Requirements** | **Mac** | **Linux** | **Windows** | **Mobile Devices** |
|---|---|---|---|---|
| **Server Side** | macOS can run Java web servers, but it is uncommon for production hosting at scale. It typically requires Apple hardware, which increases cost. There is no separate server OS license, but hardware/support costs are higher than Linux. | Linux is the most common platform for web hosting in cloud and data center environments. It is stable, secure, and easy to scale with containers and load balancing. Licensing cost is usually none or very low. | Windows Server supports reliable web hosting and can run Java services. It fits well in Microsoft ecosystems, but adds OS licensing costs and more patching/admin overhead. It can still scale when properly managed. | Mobile devices are not appropriate for hosting a scalable web server. They are designed as clients and are limited by battery, storage, and inconsistent connectivity. |

| | | | | |
|---|---|---|---|---|
| **Client Side** | macOS supports modern browsers (Safari, Chrome, Firefox) for a web-based client. Main work is responsive UI plus cross-browser testing, especially Safari. | Linux users can play through modern browsers (Firefox/Chrome). Because distributions vary, testing on common browser versions helps ensure consistent rendering. | Windows is the most common desktop client platform and supports all major browsers. Compatibility is mainly handled through standards-based web development and browser testing. | Mobile clients include iOS and Android with many screen sizes and touch input. A responsive UI is required and testing should include iOS Safari and Android Chrome. Design should account for slower networks and intermittent connectivity. |
| **Development Tools** | Java tools (JDK, Maven, Eclipse/IntelliJ, Git) work well on macOS, plus browser dev tools and Docker. If native iOS work is later needed, Xcode is required. | Linux supports the full Java toolchain, Git, and containers (Docker/Podman) with strong scripting/automation. Most tooling is open source with minimal licensing cost. | Windows supports Java development with Eclipse/IntelliJ, Maven/Gradle, Git, and Docker Desktop/WSL2. Some Windows and enterprise tool licensing may apply depending on deployment choices. | For a web client, the same web and Java tools apply plus mobile browser testing. If native apps are built, Android Studio and Xcode plus device testing add time and expertise needs. |

**Recommendations**

Analyze the characteristics of and techniques specific to various systems architectures and make a recommendation to The Gaming Room. Specifically, address the following:

1. Operating Platform: Recommend a Linux-based hosting platform for the server side of Draw It or Lose It, deployed as a web application. Linux is widely used for web hosting, offers strong stability and security tooling, and avoids per-server licensing costs. A Linux environment also aligns well with Java deployment and modern container-based workflows, making it easier to scale the application as usage grows.

2. Operating Systems Architectures: Use a 64-bit Linux architecture on x86-64 (or ARM64 where applicable) with support for multi-core processing, virtual memory, and virtualization/containers. The application should follow a layered architecture with a client tier (browser or mobile), an application/service tier (Java service layer), and a data tier (persistent storage). This separation supports maintainability and allows the service tier to scale horizontally behind a load balancer.

3. Storage Management: Use a managed relational database such as PostgreSQL or MySQL to store persistent game metadata (games, teams, players, and identifiers). If image assets are large, store them in object storage and serve them through a CDN to reduce load on the application servers. Add backups and replication for durability, and consider an in-memory cache (such as Redis) if the application needs faster reads for frequently accessed data.

4. Memory Management: On Linux, memory is managed through virtual memory with paging and demand paging, allowing processes to use more memory than physically available while the OS manages page replacement. Within the application, the Java Virtual Machine manages heap memory and uses garbage collection to reclaim unused objects. To keep memory usage predictable, the design should avoid storing large image data in memory and instead stream or fetch assets on demand, while keeping only essential game state in the JVM heap.

5. Distributed Systems and Networks: Provide the game features through RESTful APIs over HTTPS so that clients on different platforms can communicate with the same backend. Deploy multiple stateless application instances behind a load balancer to handle spikes in demand, and use the database for shared persistent state. Plan for network failures by using timeouts, retries where appropriate, and clear error handling so clients can recover from outages. Logging and monitoring should be included to trace requests across components in the distributed system.

6. Security: Protect user information by using HTTPS/TLS for all network communication, strong authentication (for example, token-based authentication), and authorization rules based on least privilege. Store passwords securely using salted hashing, validate inputs to prevent common attacks, and apply rate limiting to reduce abuse. Encrypt sensitive data at rest in the database and restrict access through firewall rules and role-based access controls. Maintain audit logs and keep dependencies updated through regular patching to reduce vulnerabilities.